

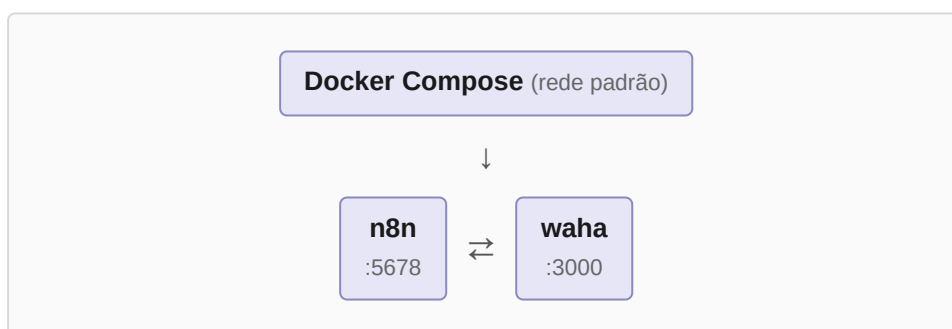
Relatório 12 - Prática: Automação de Atendimento via WhatsApp com n8n e WAHA

Lucas Augusto Nunes de Barros

Descrição da atividade

1. Configuração da Infraestrutura via Docker Compose

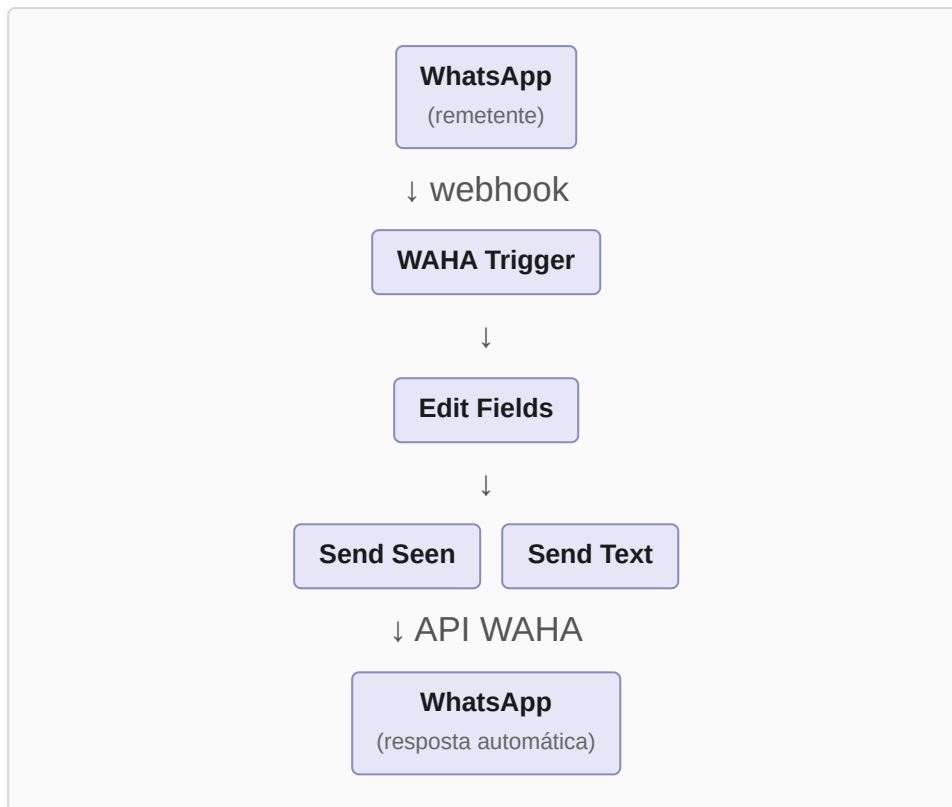
Nesta etapa prática, foi proposto o desenvolvimento de um agente local de automação de WhatsApp, integrando a API HTTP da WAHA (WhatsApp HTTP API) à plataforma de automação de fluxos n8n. Diferente da abordagem manual sugerida no material de referência — que orienta a configuração de cada container individualmente pela interface gráfica do Docker Desktop — a infraestrutura foi inteiramente descrita em um arquivo `docker-compose.yml` versionado, tornando a subida do ambiente reproduzível com um único comando (`docker compose up -d`). Os dois serviços (n8n e waha) foram colocados na mesma rede padrão do Compose, permitindo que se comuniquem entre si pelo nome do serviço (ex: `http://waha:3000`) em vez de depender da resolução de `host.docker.internal`.



Infraestrutura provisionada via Docker Compose

2. Construção do Workflow de Automação (Ping-Pong)

Após o pareamento do WhatsApp via QR Code no dashboard da WAHA, foi construído no n8n um workflow de quatro nós: um *WAHA Trigger* que recebe, via webhook, cada mensagem chegando na sessão conectada; um nó *Edit Fields* que extrai e nomeia os campos relevantes do payload recebido (remetente, texto e ID da mensagem); um nó *WAHA (Send Seen)*, que marca a mensagem como lida; e um nó *WAHA (Send Text)*, que responde automaticamente ecoando o conteúdo recebido. O fluxo foi validado de ponta a ponta: uma mensagem enviada de um número externo ao WhatsApp conectado retorna automaticamente a confirmação de leitura e a resposta de texto, sem qualquer intervenção manual.



Workflow de automação construído no n8n

Dificuldades

Ao subir os containers pela primeira vez, o n8n entrou em um ciclo de falhas (*crash loop*) por erro de permissão (EACCES) ao tentar escrever em seu diretório de dados. O container roda internamente como o usuário node (UID 1000), mas o Docker havia criado a pasta host `./n8n_data` automaticamente como `root`, já que ela não existia previamente. A correção foi criar manualmente os diretórios de dados com o usuário correto antes da primeira subida dos containers, eliminando a necessidade de ajustar permissões depois de já existirem dados gravados.

```
# criar os diretórios de dados com o usuário correto
mkdir -p n8n_data waha_data
docker compose up -d
```

A engine padrão de conexão da WAHA com o WhatsApp, *WEBJS* (baseada em automação de navegador via `whatsapp-web.js`), mostrou-se estruturalmente instável: primeiro apresentando erro 500 ao listar conversas (`getChats`), corrigido com uma atualização de versão da imagem da WAHA; em seguida, falhando de forma recorrente e irreversível ao tentar enviar mensagens de texto (`sendText`), um bug documentado em múltiplas issues abertas no repositório oficial do projeto, sem correção estável disponível até o momento desta implementação. A solução foi migrar a engine de conexão para *NOWEB*, que implementa o protocolo do WhatsApp diretamente (via biblioteca `Baileys`)

sem depender de automação de navegador — o que exigiu um novo pareamento do WhatsApp via QR Code, mas não alterou nada na configuração do workflow do n8n, já que a API da WAHA é idêntica entre engines.

```
WHATSAPP_DEFAULT_ENGINE=WEBJS # instável: getChats e sendText falhando  
WHATSAPP_DEFAULT_ENGINE=NOWEB # protocolo direto via Baileys, sem navegador
```

Mesmo após a migração de engine, o nó de envio de texto continuava, de forma intermitente, retornando sucesso (HTTP 201) sem que a mensagem chegasse de fato ao destinatário. Uma inspeção detalhada do JSON completo do workflow — não apenas da interface visual do n8n — revelou a causa raiz: o campo `chatId` do nó de envio de texto estava configurado incorretamente para usar o ID da mensagem recebida (`id_mensagem`) em vez do identificador do contato de destino (`quem_mandou`), fazendo com que a WAHA tentasse repetidamente resolver uma string de ID de mensagem como se fosse um número de telefone de destino. Após corrigir essa referência, a entrega passou a ocorrer de forma rápida e consistente em todos os testes subsequentes, inclusive em validação feita por terceiros a partir de um número diferente.

```
// nó "Send a text message" – antes  
chatId: {{ $('Edit Fields').item.json.id_mensagem }}  
  
// nó "Send a text message" – depois  
chatId: {{ $('Edit Fields').item.json.quem_mandou }}
```

Conclusões

A implementação desse agente consolidou conceitos práticos de orquestração de containers com Docker Compose e de integração entre serviços via webhooks e APIs REST, além de reforçar a importância de investigar a causa raiz de um problema a partir dos dados brutos (o JSON completo do workflow), em vez de confiar apenas na camada visual da ferramenta. A comparação entre as duas engines de conexão da WAHA também evidenciou um trade-off recorrente em bibliotecas open-source que dependem de scraping de interfaces de terceiros (WEBJS) versus implementações diretas de protocolo (NOWEB): a segunda abordagem se mostrou mais resiliente a mudanças externas no WhatsApp Web, ainda que também sujeita a bugs próprios.

Referências

[1] WAHA (WHATSAPP HTTP API). Documentação oficial.

Disponível em: <https://waha.devlike.pro/>

Acesso em 05 de agosto de 2026

[2] DEVLIKEAPRO/WAHA. Repositório oficial no GitHub — issues relacionadas aos bugs de `getChats` e `sendText` no engine WEBJS.

Disponível em: <https://github.com/devlikeapro/waha/issues/486> e
<https://github.com/devlikeapro/waha/issues/1868>

Acesso em 05 de agosto de 2026

[3] WHISKEYSOCKETS/BAILEYS. Repositório oficial no GitHub — biblioteca utilizada pelo engine NOWEB da WAHA.

Disponível em: <https://github.com/WhiskeySockets/Baileys/issues/853>

Acesso em 05 de agosto de 2026

[4] DEVLIKEAPRO/N8N-NODES-WAHA. Nó comunitário do n8n para integração com a WAHA.

Disponível em: <https://github.com/devlikeapro/n8n-nodes-waha>

Acesso em 05 de agosto de 2026